

COVERING CODES CHECKED BY THE LEAN KERNEL:

$$K_7(9, 4) \leq 1351 \text{ AND } K_2(6, 1) = 12$$

THIAGO PATZDORF

ABSTRACT. We report three results on the covering number $K_q(n, R)$, each proved in Lean 4 against Mathlib v4.34.1 and checked by the kernel, with no `sorry` and no `native.decide`. First, an explicit code of 1351 words covers $(\mathbb{Z}/7\mathbb{Z})^9$ at Hamming radius 4, so $K_7(9, 4) \leq 1351$; the best upper bound we found in the literature is 1475. Second, $K_2(6, 1) \geq 11$ follows from a short double-counting argument with no search. Third, $K_2(6, 1) = 12$, a classical value, is proved by a pruned search whose soundness is itself a theorem, split into 38 kernel decisions. The same library contains a formal proof of the sphere-covering bound and eight numerical instances of it; those instances are below the best known lower bounds and are kept only as a test of the arithmetic. Seven further codes improve the tables we consulted but are verified by computer only.

1. INTRODUCTION

The covering number $K_q(n, R)$ is the least cardinality of a code $C \subseteq A^n$, with $|A| = q$, such that every word of A^n lies at Hamming distance at most R from some word of C [1]. Upper bounds come from constructions and lower bounds from counting; the tables of Kéri [4] and the recent work of Gijswijt and Polak [3] and of Marosi [6] record the current state.

Upper bounds are certificates: a list of words and the claim that it covers. Checking the claim is a finite computation, but for q^n in the tens of millions it is done by programs that are rarely audited. Florath [2] built a Lean library of covering-code bounds whose derivations are replayed by the kernel; it formalizes the sphere-covering bound and records many cells. This note adds a certificate that the kernel checks word by word for a cell where the code is new, and a formal determination of a small classical value.

The proofs use Lean 4 [5] and Mathlib [7]. The source is <https://github.com/thiagopatzdorf/Matematica>, tag v0.3.0. A statement below is a claim only when it is the name of a declaration in that tree. The default target `CoveringLean` builds in seconds; the two search certificates form the target `CoveringHeavy`.

2. DEFINITIONS

In the development `CoveringA2` a word is a function $\text{Fin } n \rightarrow \text{ZMod } q$, distance is Mathlib’s `hammingDist`, and

$$\text{Covers } R \ C : \iff \forall x, \exists c \in C, d(x, c) \leq R.$$

In `CoveringA6` the alphabet is $\text{Fin } q$, with the same distance and the same notion of covering, and

$$\text{IsK } q \ n \ R \ k : \iff (\exists C, |C| = k \wedge \text{Covers } R \ C) \wedge \forall C, |C| < k \rightarrow \neg \text{Covers } R \ C,$$

which is the equality $K_q(n, R) = k$.

3. AN UPPER BOUND: $K_7(9, 4) \leq 1351$

Theorem 1. *There is $C \subseteq (\mathbb{Z}/7\mathbb{Z})^9$ with $|C| = 1351$ that covers at radius 4. In Lean: `CoveringKernel.K7_9_4_le_1351_kernel`, stated as*

$$\exists C : \text{Finset } (\text{Fin } 9 \rightarrow \text{ZMod } 7), \quad C.\text{card} = 1351 \wedge \text{Covers } 4 \ C.$$

Date: 2 October 2026.

The code. Words are encoded as natural numbers $\sum_i w_i 7^i$ and listed in strictly increasing order in `C1_Data_K7_9_4`; the same list is the text file `data/codes/q7_n9_R4_M1351.txt`, whose canonical SHA-256 begins `54dbdade`. The code is three cosets of a $[9, 3]_7$ linear code (1029 words) together with 322 words added greedily to cover what the cosets miss. It was found with the generator `lincov` of Marosi’s public repository `Mapika/coldcase`, which accompanies [6]; the program that added the last 322 words was not recorded. The theorem does not depend on how the code was found.

The proof. A Boolean function `go` walks the coordinates one at a time, keeping for each prefix the codewords still within reach and their remaining budget of mismatches; `go_sound` proves that `go` returning `true` implies that every word has a codeword at distance at most R . `covers_of_go` turns that into `Covers`, using that Mathlib’s distance agrees with the digit-wise count, and `cert_of_go` adds the cardinality, from a kernel check that every entry is below 7^9 and the list is strictly increasing. `go_split` lets the first four coordinates be fixed in advance, so the computation becomes $7^4 = 2401$ leaf theorems, each closed by `decide +kernel`, grouped in 95 files that the build compiles in parallel. The same pipeline accepts the binary Hamming code of length 7 and rejects a copy of it with one word replaced; both are tests in the library.

Remark. The best upper bound we found is $K_7(9, 4) \leq 1475$, by Marosi [6] (arXiv version 3, September 2026); Kéri’s tables [4] give 1843. The best lower bound we know is 264. That the bound is new is our reading of those sources on 1 October 2026, not something the kernel checks.

4. A LOWER BOUND WITHOUT SEARCH: $K_2(6, 1) \geq 11$

Lemma 2. *For distinct $a, c \in (\mathbb{F}_2)^6$, the balls of radius 1 about a and c meet in 0 or 2 words.*

Proof. At distance 1 the intersection is $\{a, c\}$; at distance 2 it is the two words between them; at distance at least 3 it is empty. In Lean the case $a = 0$ is a kernel decision over 64 words (`even_inter_zero`), and translation gives the general case (`inter_card_translate`). $\square$

Theorem 3. *Every code that covers $(\mathbb{F}_2)^6$ at radius 1 has at least 11 words (`CoveringA6.K_2_6_1_ge_11`).*

Proof. Let $K = |C|$ and let $m(y)$ be the number of codewords whose ball contains y , so $m(y) \geq 1$ and $\sum_y m(y) = 7K$. For each x put $S(x) = \sum_{y \in B(x)} m(y) = \sum_{c \in C} |B(x) \cap B(c)|$. Then $S(x) \geq 7$, and if $x \notin C$ every term is even by Lemma 2, so $S(x) \geq 8$. Summing $S(x) + [x \in C] \geq 8$ over all 64 words and using $\sum_x S(x) = 7 \sum_y m(y) = 49K$ gives $50K \geq 512$, hence $K \geq 11$. The counting is `excess_bound`, stated for any family of symmetric balls of size 7 with even pairwise intersections. $\square$

This replaces a hypothesis that the previous version of this note left open: the statement `no_cover_2_6_1_le10_uncond` is the earlier conditional one with the condition removed.

5. AN EXACT VALUE: $K_2(6, 1) = 12$

Theorem 4. *`SC.K_2_6_1_eq12` : $IsK\ 2\ 6\ 1\ 12$.*

The value is classical [8]. The upper bound is an explicit 12-word code (`code12`, encodings `0, 1, 2, 15, 28, 23, 57, 58, 59, 39, 44, 52`). For the lower bound, a search `chkN` on 64-bit masks takes a state (words still allowed, covered set, forbidden centres), picks the least uncovered word, and branches on the centres that could cover it, forbidding at each branch the siblings already tried. It prunes when the remaining words cannot cover what is left, at 7 words per centre. `chkN_sound` proves that a `true` answer means no completion exists. A translation argument lets one assume the word 63 is in the code. The search from that root is cut at depth 2 into 38 states, each refuted by `decide +kernel` in its own file. The Lean database of Florath [2] records $10 \leq K_2(6, 1) \leq 12$ at this cell.

6. THE SPHERE-COVERING BOUND

The library also proves the sphere-covering bound $q^n \leq |C| \cdot V$ (`CoveringA2.sphere_covering`), its strict form when $V \nmid q^n$, and eight numerical instances `CoveringChain.SPH.K*.lb`, among them $K_7(9, 4) \geq 221$. This was the content of version 0.2 of this note. The bound had already been formalized by Florath [2], and all eight instances are below the best lower bounds in the literature (for $K_7(9, 4)$, 264), so they are not claimed as results.

7. OTHER CODES, NOT FORMALIZED

The repository contains seven more codes, each checked by three independent programs outside Lean (Table 1). They are below the upper bounds in the tables we consulted, but none is a Lean theorem yet. For $q = 4$ our literature search was weaker than for $q \geq 5$, and $K_5(10, 4) \leq 625$ is a linear $[10, 4]_5$ code, which should still be compared with the tables of Davydov, Marcugini and Pambianco on linear covering codes.

cell	ours	previous	source of previous
$K_5(7, 2)$	500	525	Kéri
$K_4(10, 4)$	192	208	Kéri
$K_5(9, 3)$	1250	1275	Kéri
$K_5(10, 4)$	625	875	Kéri
$K_5(9, 5)$	50	55	Kéri
$K_5(9, 4)$	250	255	Kéri
$K_7(8, 3)$	1893	2337	Kéri

TABLE 1. Upper bounds verified by computer, not by the kernel.

8. WHAT THE KERNEL ACCEPTED

On 1 October 2026, on a virtual machine of type `e2-highmem-8`, Lean 4.34.1 and Mathlib v4.34.1: `lake build` finished 8942 jobs in 32s with a peak of 7.7 GB; `lake build CoveringHeavy` finished 9073 jobs in 1 h 41 min with up to eight modules in parallel, about 12.4 CPU-hours in total, at most 8.8 GB per process. `#print axioms` on Theorems 1, 3 and 4 reports `propext`, `Classical.choice` and `Quot.sound` only.

9. WHAT IS NOT A THEOREM

Novelty is a reading of the literature, not a checked fact. The seven codes of Table 1 are not Lean theorems. That the formal statements say what is intended is a human review: the reader should check the statements of Theorems 1, 3, 4 and the definitions of `Covers` and `IsK`. Two files in the tree, `A5b_Stress.lean` and `A6d_SearchHeavy.lean`, are not imported and are not claimed.

REFERENCES

- [1] G. Cohen, I. Honkala, S. Litsyn and A. Lobstein, *Covering Codes*, North-Holland, 1997.
- [2] A. Florath, *Formal foundations and proof-carrying certificates for q -ary covering codes in Lean 4*, arXiv:2606.09600, 2026.
- [3] D. Gijswijt and S. Polak, *Semidefinite lower bounds for covering codes*, arXiv:2504.01932, 2025.
- [4] G. Kéri, *Tables for bounds on covering codes*, <https://old.sztaki.hu/~keri/codes/>, 2011.
- [5] L. de Moura and S. Ullrich, *The Lean 4 theorem prover and programming language*, CADE 28, LNCS **12699**, Springer, 2021, 625–635.
- [6] M. Marosi, *New upper and lower bounds on covering codes $K_q(n, R)$ for alphabets of size $5 \leq q \leq 21$* , arXiv:2608.19872, 2026.
- [7] The Mathlib Community, *Mathlib*, <https://github.com/leanprover-community/mathlib4>, release v4.34.1.

- [8] R. G. Stanton and J. G. Kalbfleisch, *Covering problems for dichotomized matchings*, Aequationes Math. **1** (1968), 94–103.

UNIVERSIDADE DE CAXIAS DO SUL, BRAZIL

Email address: `thiagosleman@gmail.com`